

NFC 读卡接口

成都鱼住未来科技有限公司

二次开发: <http://www.yzfuture.cn>

技术支持: faq@yzfuture.cn

售后: sales@yzfuture.cn

电话: [028-82880293](tel:028-82880293)

日期	版本	说明	作者
2025/01/10	V3.0.0	支持支持身份证/港澳居民居住证/IC 卡 /银行卡/护照	TygerZH

1. 概述

本 SDK 配合本公司最新版的读卡器，支持二代身份证/港澳台居民居住证/外国人永久居留身份证/银行卡/护照/Mifare1 (S50 和 S70 的读取和写入)。

二代身份证和护照需要开通对应授权功能的读卡器才可使用。

用本 SDK 读证件需要配套我公司专用证件读卡器。

2. 硬件购买

读卡器购买：

<https://item.taobao.com/item.htm?id=610814936998>

<https://item.taobao.com/item.htm?id=611082485024>

服务器购买：

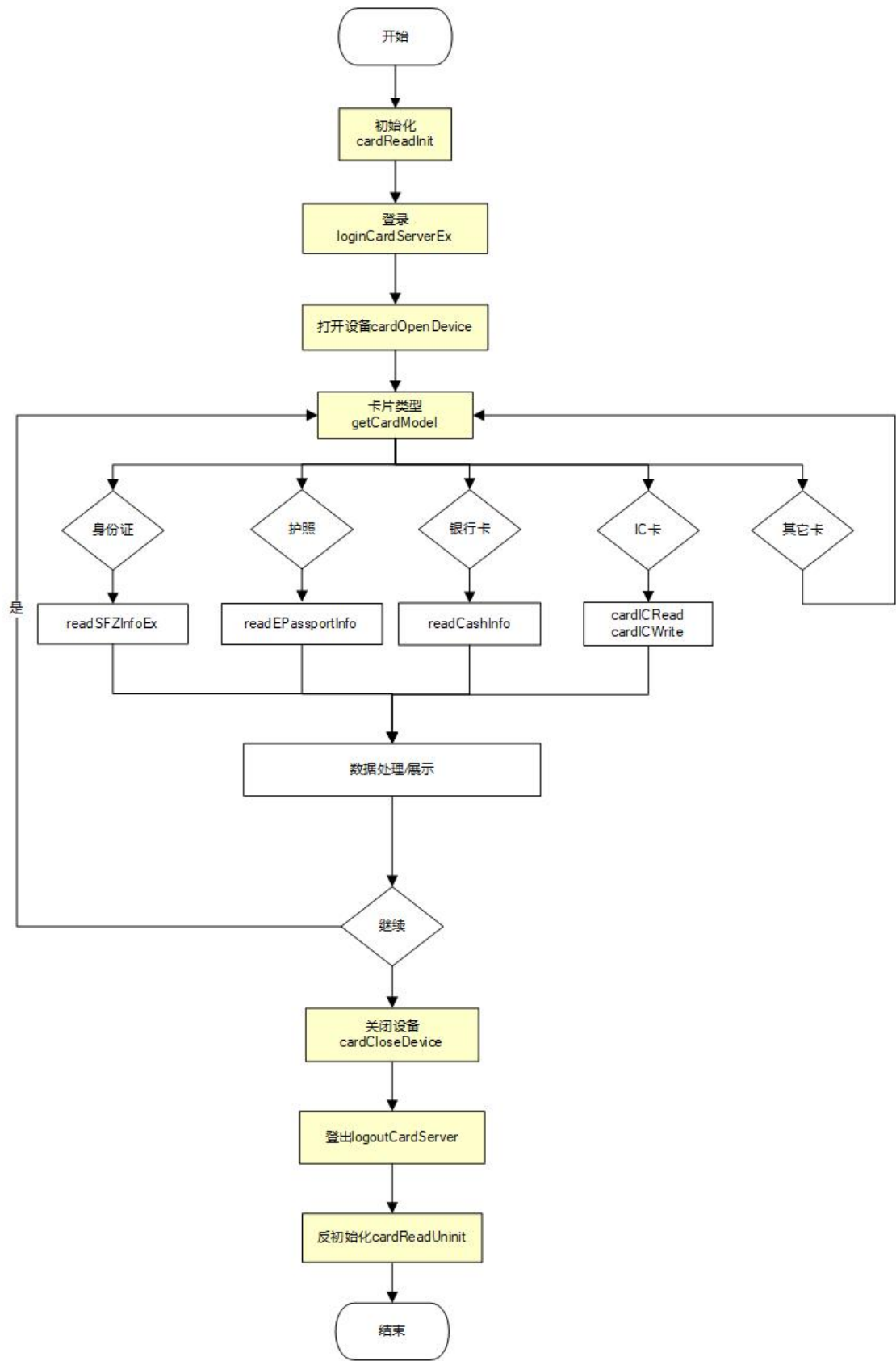
<https://item.taobao.com/item.htm?id=611230762637>

3. 接口概要

接口文件 ReadCardInfoInterface.h 中，同时需要包含 Type.h 文件。

接口头文件中带有 `// Deprecated` 标志的为过时接口，后期会逐渐停止支持

4. 调用流程



5. 接口调用样例

- 获取卡片类型

```
char        szdevSn[56] = { 0 }; // 卡片 SN 字符串
int         ndevSn = sizeof(szdevSn);
eCardModel emodel = getCardModel(hlHandle, szdevSn, &ndevSn);
switch (emodel)
{
    case SFZ_Model: // 身份证
        break;
    case EPassport_Model: // 护照
        break;
    case Cash_Model: // 银行卡
        break;
    case M1_S70_Model: // S70
        break;
    case M1_S50_Model: // S50
        break;
    default: // 其它卡
        break;
}
```

- 读二代身份证/港澳台居民居住证/外国人永久居留身份证

```
CardInfoStruct cardinfo;
bool bret = readSFZInfoEx(hlHandle, &onCardReadProgress, &cardinfo);
if (bret)
{
    if (cardinfo.etype == (char)eOldForeignerType)
    {
        // 旧版外国人居住证
    }
    else if (cardinfo.etype == (char)eNewForeignerType)
    {
        // 新版外国人居住证
    }
    else if (cardinfo.etype == eTwoGATType)
    {
        // 港澳台居住证
    }
    else
    {
        // 身份证
    }
}
```

- 读银行卡

```
char sn[64] = { 0 }; // 银行卡 sn/UID
```

```

char no[64] = { 0 }; // 银行卡号
char validity[64] = { 0 }; // 银行卡有效期
int      nsn = sizeof(sn);
int      nno = sizeof(no);
int      nvalidity = sizeof(validity);
if (readCashInfo(hlHandle, sn, &nsn, no, &nno, validity, &nvalidity))
{
    // 银行卡解码成功
}

```

- 读护照(需要单独开通授权)

```

epassportInfo epassinfo;
string          no = "护照号码";
string          birth = "六位出生日期(格式: YYMMDD)";
string          validity = "六位到期日期(格式: YYMMDD)";
if (readEPassportInfo(hlHandle,    &onCardReadProgress,    (char*)no.c_str(),
(char*)birth.c_str(), (char*)validity.c_str(), &epassinfo))
{
    // 护照解码成功
}

```

- 读写 IC 卡

```

int      nsector = 0; // 扇区
int      nblock = 2; // 块
char szapwd[6] = { 0xff, 0xff, 0xff, 0xff, 0xff, 0xff }; // A 密钥
char szbpwd[6] = { 0xff, 0xff, 0xff, 0xff, 0xff, 0xff }; // B 密钥
char szread[16];
char szwrite[16] = {0xff};
memset(szread, 0, sizeof(szread));
if (cardICRead(hlHandle, szapwd, szbpwd, nsector, nblock, szread))
{
    // 读扇区成功
}
if (cardICWrite(hlHandle, szapwd, szbpwd, nsector, nblock, szwrite))
{
    // 写扇区成功
}

```

6. 回调

- typedef void(__stdcall *cardReadProgress)(unsigned int nProgress, YZWLHandle nhandle);
 返回身份证和护照读卡进度 (其它卡片无返回)
 一共 20 步。

7. 结构体

- 卡片类型

```
typedef enum eCardModel
{
    Unknown_Model,    // 未知卡片
    SFZ_Model,        // 身份证
    EPassport_Model,  // 护照
    Cash_Model,       // 银行卡
    M1_S70_Model,     // S70
    M1_S50_Model,     // S50
    M1_CPU_Model,     // 其它 CPU 卡，暂不支持操作
    M1_Ultralight_Model, // Ultralight 卡
    M1_DESFire_Model  // DesFire 卡，暂不支持操作
};
```

- 二代证格式类型

```
typedef enum _eCardFormatType
{
    eTwoIDType = 'I', // 身份证
    eTwoGATType = 'J', // 港澳台居民居住证
    eOldForeignerType = 'L', // 外国人永久居留身份证
    eNewForeignerType = 'Y', // 外国人永久居留身份证(新版)
}eCardFormatType;
```

- 身份证和港澳台居住证解码结构

```
typedef struct _TwoIdInfoStruct
{
    char arrName[30];           //姓名 UNICODE
    char arrSex[2];             //性别 UNICODE
    char arrNation[4];          //民族 UNICODE
    char arrBirthDay[16];       //出生日期 UNICODE YYYYMMDD
    char arrAddress[70];        //住址 UNICODE
    char arrNo[36];              //身份证号码 UNICODE
    char arrSignedDepartment[30]; //签发机关 UNICODE
    char arrValidityPeriodBegin[16]; //有效期起始日期 UNICODE YYYYMMDD
    char arrValidityPeriodEnd[16];   //有效期截止日期 UNICODE YYYYMMDD 有效期
    //为长期时存储“长期”

    char arrOtherNO[18]; // 通行证类号码
    char arrSignNum[4];   // 签发次数
    char arrRemark1[6];   // 预留区
    char arrType[2];      // 证件类型标识
};
```

```

char arrRemark2[6];    // 预留区

unsigned char arrPhoto[1024];    //照片信息
unsigned char arrFingerprint[1024]; //指纹信息
}TwoIdInfoStruct;

```

- 外国人永久居住证旧版

```

typedef struct _ForeignerInfoOld // 外国人永久居住证
{
    char arrEnName[120];          //英文名
    char arrSex[2];               //性别 UNICODE
    char arrNo[30];               //15 个字符的居住证号码 UNICODE
    char arrCountry[6];           //国籍 UNICODE GB/T2659-2000
    char arrName[30];             //中文姓名 UNICODE 如果没有中文姓名，则全为
0x0020
    char arrValidityPeriodBegin[16]; //签发日期 UNICODE YYYYMMDD
    char arrValidityPeriodEnd[16];   //终止日期 UNICODE YYYYMMDD
    char arrBirthDay[16];           //出生日期 UNICODE YYYYMMDD

    char arrVersion[4]; // 版本号
    char arrSignedDepartment[8];    //签发机关代码 UNICODE 证件芯片内不存储
签发机关
    char arrType[2];               // 证件类型标识
    char arrRemark2[6];            // 预留区

    unsigned char arrPhoto[1024];    //照片信息
    unsigned char arrFingerprint[1024]; //指纹信息
}ForeignerInfoOld;

```

- 外国人永久居住证新版

```

typedef struct _ForeignerInfoNew // 外国人永久居住证新版
{
    char arrName[30];             //姓名 UNICODE
    char arrSex[2];               //性别 UNICODE
    char arrNation[4];            //民族 UNICODE
    char arrBirthDay[16];         //出生日期 UNICODE YYYYMMDD
    char arrEnName[70];           //外文姓名 UNICODE
    char arrNo[36];               //身份证号码 UNICODE
    char arrSignedDepartment[30]; //签发机关 UNICODE
    char arrValidityPeriodBegin[16]; //有效期起始日期 UNICODE YYYYMMDD
    char arrValidityPeriodEnd[16];   //有效期截止日期 UNICODE YYYYMMDD 有效期
为长期时存储“长期”

    char arrOtherNO[18]; // 通行证类号码
    char arrSignNum[4];      // 签发次数

```



```

char arrCountry[6];          //国籍 UNICODE GB/T2659-2000
char arrType[2];             // 证件类型标识
char arrRemark2[6];          // 预留区

unsigned char arrPhoto[1024]; //照片信息
unsigned char arrFingerprint[1024]; //指纹信息
}ForeignerInfoNew;

```

- 身份证+港澳台居住证+外国人永久居住证

```

typedef struct _CardInfoStruct
{
    char etype; // 证件类型，用来区分下面哪个结构有效
    union
    {
        TwoIdInfoStruct twoId; // 身份证/港澳台居民居住证
        ForeignerInfoOld foreigner; // 旧版外国人永久居住证
        ForeignerInfoNew newForeigner; // 新版外国人永久居住证
    } info;
}CardInfoStruct;

```

- 护照

```

typedef struct _strEpassportInfo
{
    char arrPaperType[2]; // documentCode:mrz 前两位 证件类型(缩写)
    char arrTypeFullName[99]; // 证件类型(全称)
    char arrSignedDepartment[99]; // 签发国家或签发机构
    char arrENName[48]; // 按照机读区
    char arrPrimaryName[48]; // 英文姓
    char arrSecondName[48]; // 英文名
    char arrOtherName[99]; // 其它姓名
    char arrName[48]; // utf8 型中文名
    char arrNo[16]; // 证件号码
    char arrDocumentID[16]; // 个人编码
    char arrCountry[3]; // 国籍编码
    char arrBirthDay[6]; // 生日 YYMMDD
    char cSex; // 性别 M/F
    char arrValidityPeriodEnd[6]; // 有效期 YYMMDD
    char arrFaceJpg[100 * 1024]; // 人脸 jpg 图片
    int nFaceLen; // 人脸图片长度
}epassportInfo;

```

8. 解码接口

为支持不同语言开发，接口统一为标准 C 接口。

- 初始化: `cardReadInit`

```
void __stdcall cardReadInit();
```

SDK 初始化操作，只需要在程序最开始构造的时候调用一次，用来初始化相关初始参数。

- 登录: `loginCardServerEx`

```
bool __stdcall loginCardServerEx(char* szip, int nport, int &nerr);
```

登录解码服务器，不使用 appkey 进行管理的话，可直接调用此接口。

`szip`: 中心解码服务器地址 (eg: id.yzfuture.cn)

`nport`: 中心解码服务器开放商品 (eg: 443)

`nerr`: 错误码

返回值:

成功 - true

失败 - false

- 打开设备: `cardOpenDevice`

```
YZWLHandle __stdcall cardOpenDevice(int nouttime, int &nerr, int nDeviceNo);
```

打开读卡器硬件设备。

`nouttime`: 超时时间 (秒)

`nerr`: 错误码

`nDeviceNo`: 读卡器序号，默认为 0 (为解决同时有多个相同读卡器的情况，可以选其中某一个为读卡设备) (如果 `setDeviceType` 设置为离线读卡器的话，`nDeviceNo` 应该为 1001~1016 中的某一值)

返回值:

成功 - 读卡器句柄

失败 <= 0

- 卡片类型: `getCardModel`

```
eCardModel __stdcall getCardModel(YZWLHandle nDeviceHandle, char* szsn, int* nsn)
```

获取卡片类型

`nDeviceHandle`: 打开的设备句柄

`szsn`: 卡片 UID (身份证为 SN/护照为临时的可变识别码)

`nlen`: 返回的 SN 码字符串长度，字节为单位 (传入时需要赋值 `szsn` 的字节空间大小，如果小于实际 SN 码长度，则返回失败，空间大小需大于 32 字节)

返回值:

卡片类型枚举值

- 读二代证: `readSFZInfo`

```
bool __stdcall readSFZInfo(YZWLHandle nDeviceHandle, cardReadProgress cardCB,
char* szbase, int* nlen, char* ctype, bool boldSdt)
```

读身份证、港澳居住证、外国人居住证。

nDeviceHandle: 打开的设备句柄

cardCB: 回调的读卡进度

szbase: 身份证原始数据，大小不能小于 4096

nlen: 原始数据大小 (传入 szbase 空间大小，传出实际使用大小)

ctype: 二代证格式类型，数据同 [eCardFormatType](#)

boldSdt: 兼容旧离线读卡器，默认为 false 即可

返回值:

True - 成功

False - 失败

- **读二代证: readSFZInfoEx**

```
bool __stdcall readSFZInfoEx(YZWLHandle nDeviceHandle, cardReadProgress
cardCB, CardInfoStruct* cardinfo)
```

在 [readSFZInfo](#) 的基础上做的扩展，用结构体方式解析对应字段。

nDeviceHandle: 打开的设备句柄

cardCB: 回调的读卡进度

cardinfo: 解码后的结果保存在这个结构体里 (unicode 格式)

返回值:

True - 成功

False - 失败

- **二代证头像: decodeCardImage**

```
bool __stdcall decodeCardImage(unsigned char* srcimage, char* outimage, int&
outlen);
```

将读出来的身份证信息解码成 bmp 格式

srcimage: 读出来的头像信息原数据

outimage: 解码出来的图片数据，内存空间由用户自己管理，不得小于 40K

outlen: 传入时为 outimage 实际大小，传出时为实际图片大小

返回值:

True - 成功

False - 失败

- **二代证 DN: cardGetCardDN**

```
bool __stdcall cardGetCardDN(YZWLHandle nDeviceHandle, char* szsn, int &nlen)
```

获取身份证 DN 码 (离线读卡器返回数据为空)

nDeviceHandle: 打开的设备句柄

szsn: DN 码

nlen: 返回的 DN 码字符串长度，字节为单位 (传入时需要赋值 szsn 的字节空间大小，如果小于实际 DN 码长度，则返回失败，空间大小需大于 64 字节)

返回值:

True - 成功

False - 失败

- 读护照: readEPassportInfo

```
bool __stdcall readEPassportInfo(YZWLHandle nDeviceHandle, cardReadProgress
cardCB, char* no, char* birth, char* validity, epassportInfo* cardinfo)
```

读取护照信息 (需要读卡器有护照读取权限)

nDeviceHandle: 打开的设备句柄

cardCB: 回调的读卡进度

no: 护照号 (读取前需要先传进去)

birth: 六位出生日期 (格式: YYMMDD) (读取前需要先传进去)

validity: 六位有效期 (格式: YYMMDD) (读取前需要先传进去)

cardinfo: 解码后的结果保存在这个结构体里

返回值:

True - 成功

False - 失败

- 读银行卡: readCashInfo

```
bool __stdcall readCashInfo(YZWLHandle nDeviceHandle, char* sn, int* nsn,
char* no, int* nno, char* validity, int* nvalidity)
```

读取银行卡信息

nDeviceHandle: 打开的设备句柄

sn: 读取到的银行卡 UID

nsn: 读取到的银行卡 UID 长度

no: 读取到的银行卡号

nno: 读取到的银行卡号长度

validity: 读取到的银行卡有效期

nvalidity: 读取到的银行卡有效期长度

返回值:

True - 成功

False - 失败

- 读 IC 卡 (S50/S70) : cardICRead

```
bool __stdcall cardICRead(YZWLHandle nDeviceHandle, char szpwdA[6], char
szpwdB[6], int nsector, int nblock, char szread[16])
```

读取 S50/S70 卡对应扇区数据

nDeviceHandle: 打开的设备句柄

nsector: 需要读取的扇区编号 (以 0 开始)

nblock: 对应扇区的块编号 (0~3)

szpwdA: 扇区对应的 A 密钥

szpwdB: 扇区对应的 B 密钥

szread: 读取到的 16 字节长度数据

返回值:

True - 成功

False - 失败

- 写 IC 卡 (S50/S70) : cardICWrite

```
bool __stdcall cardICWrite(YZWLHandle nDeviceHandle, char szpwdA[6], char
```

```
szpwdB[6], int nsector, int nblock, char szwrite[16])
```

对 S50/S70 卡对应扇区数据写入

nDeviceHandle: 打开的设备句柄

nsector: 需要写入的扇区编号 (以 0 开始)

nblock: 对应扇区的块编号 (0~3, 一般 0 号扇区 0 号块禁止写入, 会造成锁卡无法恢复)

szpwdA: 扇区对应的 A 密钥

szpwdB: 扇区对应的 B 密钥

szwrite: 需要写入的 16 字节长度数据

返回值:

True - 成功

False - 失败

- **读取流水号: cardGetSerialNumber**

获取当次流水号, 可用于后期对账 (离线读卡器返回数据为空)

返回值:

字符串形式的流水号, 如果解码在鉴权前就失败, 则流水号可能为空

- **蜂鸣: cardBeep**

```
bool __stdcall cardBeep(YZWLHandle nDeviceHandle);
```

读卡器蜂鸣一次 (10ms) (离线读卡器有可能无效)

nDeviceHandle: 打开的设备句柄

返回值:

True - 成功

False - 失败

- **错误码: cardGetLastErrorCode**

```
int __stdcall cardGetLastErrorCode(YZWLHandle nDeviceHandle);
```

获取最后一次出错误码

nDeviceHandle: 打开的设备句柄

返回值:

True - 成功

False - 失败

- **错误信息: cardGetErrorInfo**

```
const char* __stdcall cardGetErrorInfo(YZWLHandle nDeviceHandle, int
&nlen, int nErrorCode);
```

根据传入的错误代码查找对应的错误信息

nDeviceHandle: 打开的设备句柄

nlen: 返回错误信息的长度

nErrorCode: 需要查找的错误代码

返回值:

查找出错信息

- **关闭读卡器: cardCloseDevice**

```
void __stdcall cardCloseDevice(YZWLHandle nDeviceHandle);
```

关闭读卡器设备

nDeviceHandle:打开的设备句柄

返回值:

无

- 登出: logoutCardServer

```
void __stdcall logoutCardServer();
```

登出服务器操作, **离线读卡器可以不调用此接口**

- 反初始化: cardReadUninit

```
void __stdcall cardReadUninit();
```

反初始化操作, 最后程序析构的时候调用

9. 错误码

调用 cardGetLastErrorCode 获取错误码

调用 cardGetErrorInfo 获取对应错误信息

10. 关于二代证民族代码

民族代码部分对应关系在 sdk->d11->CardInfo.ini 中

11. 关于二代证男女代码

民族代码部分对应关系在 sdk->d11->CardInfo.ini 中

12. 关于外国及地区代码

对应关系在 sdk->d11->CardInfo.ini 中